CIM-Hub Carsharing Tanzania

Technical Report

App-Supported Carsharing Service for Tanzania

Supervised by:

Prof. Dr. Simone Strambach Prof.
Dr. Gabriele Taentzer
Rebecca Saggau

Philipps Universität Marburg

Students:

Ahmed Elmesery (Mat.-Nr. 3873229)
Hikmat Samadov (Mat.-Nr. 3965696)
Sumeet Kumar (Mat.-Nr. 3848171)
Mohamed Fakhr (Mat.-Nr. 3791264)

Table of Contents

Table of Contents....2
1. Requirements Analysis....3
    1.1 C1 — Definition of the Target System....3
    1.2 C2 — Functional Requirements....4
    1.3 C3 — Non-Functional Requirements....8
2. System Design....11
    2.1 D1 — High-Level Architecture....11
    2.2 D2 — Technology Inventory....15
    2.3 D3 — Detailed Design....17
    2.4 D6 — Data Model....22
3. Quality Assurance....26
    3.1 E1 — Test Plan....26
    3.2 E2 — Test Report....27
4. Final Report....30
    4.1 A2 — Summary....30
    4.2 E3 — Example Application....31
    4.3 F1 — User Documentation....32
    4.4 F2 — Developer Documentation....33
5. Code Documentation Standards....35
    Documentation Principles....35
    Applied Practice....35
    Naming and Readability....35
6. Technical Retrospective....36
    What Worked Well....36
    What Was Challenging....36
    What We Would Do Differently....36
    Outlook....37
Appendix: Key References....38
    Backend Documentation....38
    Backend Key Source Files....38
    Frontend Repositories....38

1. Requirements Analysis

1.1 C1 — Definition of the Target System

System Overview

Kamili Drive is a university-focused car-sharing platform consisting of a cross-platform Flutter mobile client (Android / iOS / Desktop) and a set of Java Spring Boot microservices deployed on Google Kubernetes Engine. The application enables drivers to publish trips and passengers to search for, book, and cancel rides. The backend exposes an HTTP/JSON API gateway and provides end-to-end support for ride offering, trip discovery, booking management, and mutual reviews.

Client (Flutter) — Deployment Targets

- Android: API Level 21+ (Android 5.0 Lollipop or higher)
- iOS: iOS 13.0+
- Desktop: Windows, macOS, Linux via Flutter compilation (development / testing)
- RAM: 2 GB recommended; ~50 MB installation; internet + GPS required

Backend — Deployment Environment

The backend is hosted on Google Cloud Platform (GKE cluster). The API gateway is exposed via a GCE Ingress. Services communicate internally via Kubernetes ClusterIP services, and all external traffic enters through the Spring Cloud Gateway.

Backend — Service Inventory

Service	Base Path	Responsibility
api-gateway	http://35.186.208.67	Request routing, JWT validation, admin enforcement, request-ID propagation
auth-service	/auth-service/...	JWT issuance, signup/login, token validation, password reset
trip-service	/trip-service/...	Trip creation, geospatial search, booking lifecycle, OSRM enrichment, admin reports
user-service	/user-service/...	User profiles, driver/passenger profiles, vehicles, driver documents
review-service	/review-service/...	Mutual reviews between drivers and passengers
notification-service	/notification-service/...	Push notifications via Firebase Admin SDK **[Under Development]**

External Services

Service	URL	Purpose
OpenStreetMap Nominatim	nominatim.openstreetmap.org	Address autocomplete and geocoding

Service	URL	Purpose
OpenStreetMap Tiles	via flutter_map	Visual map rendering
OSRM Routing	router.project-osrm.org	Route geometry, distance, and duration enrichment for trips

Primary Actors

Actor	Responsibilities
Guest	Access public endpoints (login, signup)
Passenger	Search trips, create bookings, cancel bookings, leave reviews, view booking history
Driver	Offer trips, manage trip lifecycle, accept/reject bookings, register vehicles, leave reviews
Admin	Moderate entities, query audit logs via admin endpoints

1.2 C2 — Functional Requirements

Frontend Functional Requirements

ID	Requirement	Priority
FR-01	Users can register using email, password, name, phone number, age, and role (passenger or driver).	REQUIRED
FR-02	Users can log in using email and password and receive a JWT token.	REQUIRED
FR-03	Users can log out, which clears locally stored authentication data.	REQUIRED
FR-04	Passengers can search for trips by specifying source, destination, date, and seat count.	REQUIRED
FR-05	The system provides address autocomplete suggestions while the user enters source and destination.	REQUIRED
FR-06	Users can interact with a map to view locations, select a location, and retrieve current device location.	REQUIRED
FR-07	Search results display trip details including route, departure time, available seats, and driver info.	REQUIRED
FR-08	Passengers can book a trip by selecting it and confirming with the supported payment method.	REQUIRED
FR-09	The currently supported payment option is cash.	REQUIRED
FR-10	Passengers can view their bookings, including upcoming and past trips.	REQUIRED

ID	Requirement	Priority
FR-11	Passengers can cancel a booking before the trip date without driver confirmation.	REQUIRED
FR-12	Drivers can register a vehicle by entering make/model, license plate, vehicle type, and seating capacity.	REQUIRED
FR-13	Vehicle registration does not currently include official license verification.	REQUIRED (noted limitation)
FR-14	Drivers can create a new trip specifying vehicle, route, departure time, and available seats.	REQUIRED
FR-15	Drivers can view a list of trips they have created.	REQUIRED
FR-16	Drivers can view booking status and remaining or occupied seats for their trips.	REQUIRED
FR-17	Drivers can cancel or withdraw a trip they created.	REQUIRED
FR-18	The application stores authentication tokens securely on the device.	REQUIRED
FR-19	The application automatically includes the JWT bearer token in authenticated API requests.	REQUIRED
FR-20	On HTTP 401, the application clears stored credentials and redirects the user to login.	REQUIRED
FR-21	Future: Tanzanian payment gateways (e.g., AzamPay) support.	OPTIONAL
FR-22	Future: vehicle or driver-license verification workflows.	OPTIONAL

Backend Functional Requirements

Requirements derived directly from codebase analysis, organized by domain. All requirements are considered “REQUIRED".

FR-AUTH — Authentication & Authorization

ID	Requirement
FR-AUTH-01	Users shall register as either DRIVER or PASSENGER (userType selected at signup).
FR-AUTH-02	Signup shall return a JWT token immediately upon success.
FR-AUTH-03	All emails shall be normalized to lowercase before storage.
FR-AUTH-04	If user-service profile creation fails during signup, the auth account shall be rolled back.
FR-AUTH-05	Password reset tokens expire after 1 hour (3,600,000 ms).
FR-AUTH-06	JWT validation is performed by auth-service on behalf of the gateway for every protected request.

FR-DRIVER — Driver Management

ID	Requirement
FR-DRIVER-01	Newly registered drivers start with status PENDING.
FR-DRIVER-02	Only admins can approve (PENDING → ACTIVE) or reject (PENDING → REJECTED) drivers.
FR-DRIVER-03	Admins can bulk-approve or bulk-reject multiple drivers in a single request.
FR-DRIVER-04	A driver can register at most 5 vehicles.
FR-DRIVER-05	Vehicle license plates must be unique and are normalized to uppercase.

FR-TRIP — Trip Lifecycle

ID	Requirement
FR-TRIP-01	Driver creates a trip; route geometry is calculated via OSRM (distance, duration, GeoJSON).
FR-TRIP-02	Trip timezone is auto-detected from the pickup location coordinates.
FR-TRIP-03	Trip timestamps are stored in UTC, truncated to seconds.
FR-TRIP-04	A trip requires a minimum of 1 seat.
FR-TRIP-05	Trip cancellation cascades — all non-cancelled bookings are automatically cancelled.
FR-TRIP-06	Only the driver who created the trip can cancel it.

Trip Status Machine:

CREATED → AVAILABLE → IN_PROGRESS → COMPLETED | Any status → CANCELLED

FR-BOOKING — Booking Workflow

ID	Requirement
FR-BOOK-01	A passenger cannot book the same trip twice (duplicate check on non-cancelled bookings).
FR-BOOK-02	Booking is created in REQUESTED state and requires driver confirmation.
FR-BOOK-03	Driver can only accept/reject bookings in REQUESTED state.
FR-BOOK-04	Rejected bookings refund the reserved seats back to the trip.
FR-BOOK-05	Passengers can cancel REQUESTED or CONFIRMED bookings; seats are refunded.
FR-BOOK-06	Price per seat and estimated price are locked in at booking time.

Booking Status Machine:

REQUESTED → CONFIRMED → IN_PROGRESS → COMPLETED

REQUESTED → REJECTED

REQUESTED or CONFIRMED → CANCELLED

FR-SEARCH — Trip Search

ID	Requirement
FR-SEARCH-01	Search by proximity to source location (default radius: 5 km).
FR-SEARCH-02	Search by proximity to destination location (default radius: 5 km).
FR-SEARCH-03	Combined route-match search filtering by: time window, seats, price range (min/max), car type.
FR-SEARCH-04	Destination proximity filtering is done in-memory via Haversine (MongoDB limitation: single $geoNear per query).

FR-REVIEW — Review System

ID	Requirement
FR-REV-01	Reviews require both the trip and booking to be in COMPLETED status.
FR-REV-02	Only one review per trip/booking combination is allowed.
FR-REV-03	Ratings must be between 1 and 5 (inclusive).
FR-REV-04	Reviews are created with status PUBLISHED by default.
FR-REV-05	Admins can flag, remove, or re-publish reviews.
FR-REV-06	Average rating is calculated only across PUBLISHED reviews.

Review Status Machine:

PUBLISHED ↔ FLAGGED | PUBLISHED or FLAGGED → REMOVED

FR-NOTIF — Notifications

ID	Requirement
FR-NOTIF-01	Users receive push notifications for booking and trip events via Firebase Admin SDK. **[Service Under Development]**

FR-ADMIN — Admin & Reporting

ID	Requirement
FR-ADMIN-01	Admin can manage (view, approve, reject) drivers, passengers, vehicles, trips, bookings, and reviews via dedicated admin endpoints.
FR-ADMIN-02	Admin can query immutable audit logs via reporting endpoints.
FR-ADMIN-03	All admin list endpoints support pagination (Spring Data Pageable).

Role × Permission Matrix

Action	Public	Passenger	Driver	Admin
Register / Login	✓	✓	✓	✓
Search trips	✓	✓	✓	✓
Offer a trip			✓	
Cancel own trip			✓	
Book a ride		✓		
Cancel own booking		✓		
Accept / reject booking			✓ (own trips)	
Submit review of driver		✓		
Submit review of passenger			✓	
Approve / reject drivers				✓
Moderate reviews				✓
View audit logs / reports				✓
Manage all trips / bookings				✓

1.3 C3 — Non-Functional Requirements

Performance

ID	Requirement	Measure
NFR-P1	Address autocomplete shall not trigger on every keystroke.	300 ms debounce delay before sending a geocoding request.
NFR-P2	Network operations shall not block the UI thread.	All API operations use async / await.
NFR-P3	Remote requests shall fail in bounded time if the backend is unreachable.	Connect, send, receive timeout: 30 seconds.
NFR-P4	Trip search shall provide fallback if the primary matching endpoint fails.	Fallback to alternative route-search logic with client-side filtering.

Security

ID	Requirement	Measure
NFR-Sec-1	Authenticate requests using JWT and reject missing/invalid tokens (401).	Gateway validates JWTs by delegating to auth-service.
NFR-Sec-2	Prevent header spoofing for identity headers passed downstream.	Gateway strips incoming X-User-* headers before processing.

ID	Requirement	Measure
NFR-Sec-3	Restrict admin endpoints to admin role.	AdminAuthorizationFilter enforces admin-path rules at gateway; AdminGuard at service.
NFR-S1	Authentication tokens shall never be stored in plain text (client).	Stored using platform-encrypted Flutter Secure Storage.
NFR-S4	Expired or invalid sessions shall be invalidated locally.	Stored credentials cleared automatically on HTTP 401.

Backend — Quantifiable Constraints (from Codebase)

ID	Requirement	Source
NFR-01	Password reset tokens must expire within 1 hour.	AuthService.java:160
NFR-02	Vehicles per driver must not exceed 5.	VehicleController
NFR-03	Trip seat count minimum: 1.	@Min(1L) on Trip model
NFR-04	Booking seat request minimum: 1.	@Min(1) on Booking DTO
NFR-05	Review rating range: 1–5.	@Min(1) @Max(5) on Review model
NFR-06	Price per seat: >= 0.	@PositiveOrZero
NFR-07	Default search radius: 5.0 km.	TripGeospatialService
NFR-08	All timestamps stored in UTC, truncated to seconds.	OfferRideService
NFR-09	Geospatial queries backed by MongoDB 2dsphere index on source + destination.	MongoConfig.java
NFR-10	All admin list endpoints must support pagination (Spring Data Pageable).	Admin controllers
NFR-11	Vehicle license plates must be globally unique.	MongoDB unique index

Availability, Scalability & Observability

- NFR-Avail-1: Provide readiness and liveness endpoints for Kubernetes probing (Spring Boot Actuator).
- NFR-Avail-2: Deploy microservices with resource requests/limits (100m CPU / 256Mi requests; 500m / 512Mi limits).
- NFR-Scale-1: Services run behind Kubernetes Services for load balancing; HPAs exist.
- NFR-Obs-1: Correlate requests end-to-end using X-Request-Id.
- NFR-Obs-2: Expose Prometheus metrics; integrate with monitoring stack (Prometheus, Grafana, Loki, Fluent Bit).
- NFR-Audit-1: Persist immutable audit logs for significant state changes in audit_logs collection.

Maintainability

- NFR-Maint-1: Modular services with shared common module for cross-cutting concerns (backend).

- NFR-M1: Business logic separated from UI code via layered architecture (frontend).
- NFR-M2: HTTP communication centralized through Dio client (frontend) / Spring Cloud Gateway (backend).
- NFR-M3: JSON serialization generated rather than implemented manually (json_serializable / Lombok).

2. System Design

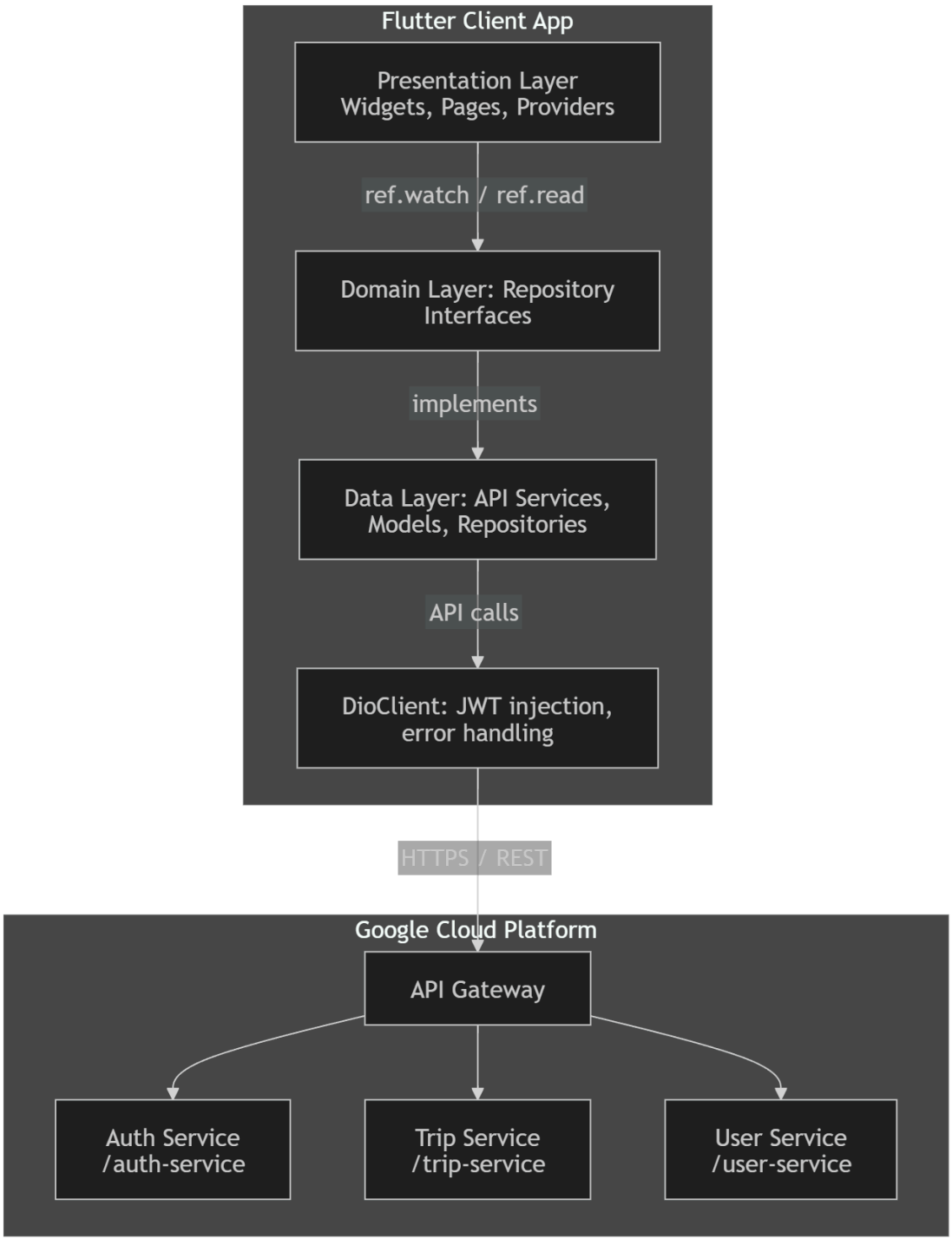
2.1 D1 — High-Level Architecture

Architectural Style

Kamili Drive follows a client-server architecture. The Flutter application acts as the client and communicates with remote microservices via an API gateway using HTTPS/JSON. Within the client, the codebase is organised using a feature-oriented layered architecture influenced by Clean Architecture principles. The backend follows a microservices architecture deployed on Kubernetes.

Frontend Architectural Layers

- Presentation Layer: Flutter widgets, screens, forms, and UI-specific state handling.
- State Management / Provider Layer: Riverpod manages application state, async loading, dependency injection, and interaction between UI and data sources.
- Data / Repository Layer: Encapsulates data access; structured methods for authentication, trips, bookings, users, vehicles, and geolocation.
- Network and Storage Layer: HTTP via Dio, and secure local session storage via flutter_secure_storage.



Client–Backend Interaction Flow

1. The user interacts with the UI.
2. The UI reads or triggers Riverpod providers and state notifiers.
3. Providers call service or repository methods.
4. Services use a centralized Dio client to communicate with backend endpoints.

5. Responses are deserialized into typed model objects and returned to the UI.

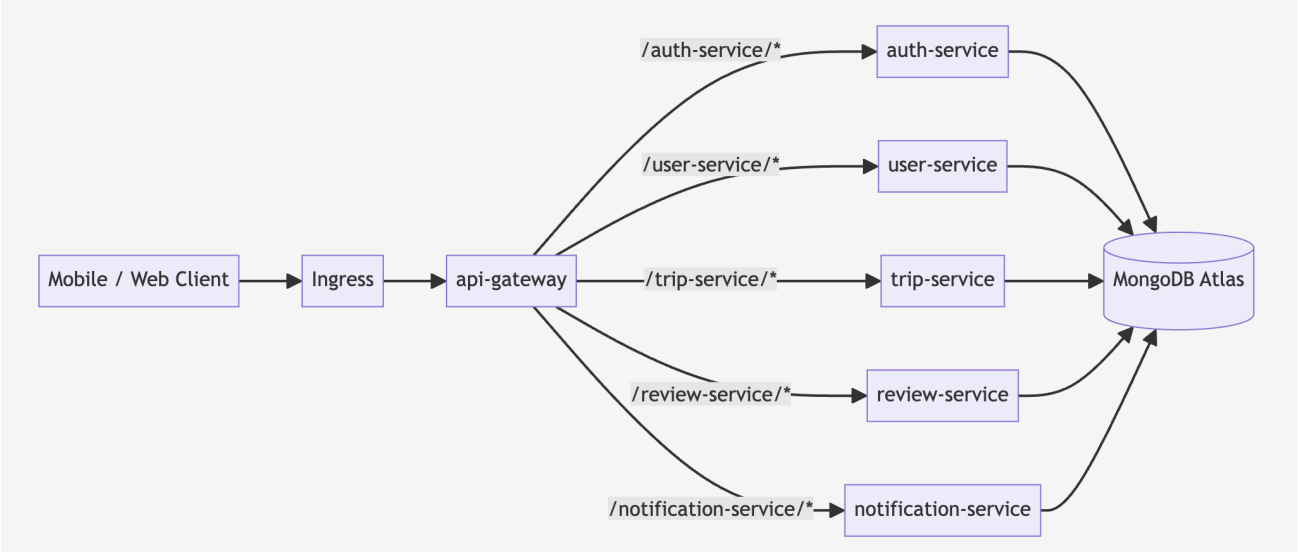
Component Interfaces

From	To	Interface
Flutter UI widgets	Riverpod providers	ref.watch() / ref.read()
Providers / state notifiers	Services / repositories	method calls
Services / repositories	Dio client	get(), post(), put(), delete()
Dio client	Backend services	HTTP/JSON over TCP
Secure storage wrapper	Dio client / auth flow	token read / write / clear

Backend — Key Services

Service	Responsibility	Key Evidence
api-gateway	Single entry point, routing, JWT validation, admin path enforcement, request-ID propagation	JwtValidationFilter.java, AdminAuthorizationFilter.java, application.yml
auth-service	JWT issuance (signup/login), token validation, password reset	AuthController.java, AuthService.java, JwtService.java
user-service	User profiles, driver/passenger profiles, vehicles, documents	model/*, vehicle/model/Vehicle.java
trip-service	Trip offer, geospatial search, booking lifecycle, admin reports	OfferRideService.java, TripGeospatialService.java, ReportsController.java
review-service	Mutual reviews between drivers and passengers	model/Review.java
notification-service	Push notifications via Firebase Admin SDK **[Under Development]**	Module present in parent pom.xml; implementation not yet complete
common	Shared DTOs, error handling, audit logging, request-ID filter	ApiResponseWrapper.java, GlobalExceptionHandler.java, AuditService.java, RequestIdFilter.java

Backend — Component Diagram



Backend — Gateway Routing

Path Prefix	Target Service (StripPrefix=1)
/user-service/**	http://user-service:8080
/auth-service/**	http://auth-service:8080
/trip-service/**	http://trip-service:8080
/review-service/**	http://review-service:8080
/notification-service/**	http://notification-service:8080

Security Model

JWT tokens are issued by auth-service and presented by clients via Authorization: Bearer <token>. The gateway validates JWTs by delegating token validation to auth-service. After validation, the gateway injects X-User-Email, X-User-Id, and X-User-Role headers for downstream services. Incoming X-User-* headers are stripped before processing to prevent spoofing. Admin endpoints are protected by path-based rules at the gateway (AdminAuthorizationFilter) and additionally enforced at service level via AdminGuard.requireAdmin.

RBAC Matrix

Layer	Principal	Scope	Permission Model
Kubernetes (RBAC)	ServiceAccount spring-cloud	Namespace carsharing	Role grants get/list/watch on pods, services, endpoints
Application (Gateway)	Caller with JWT	All gateway routes	Public paths bypass JWT; protected paths require validation; admin paths require role ADMIN

Layer	Principal	Scope	Permission Model
Application (Services)	Downstream service reading headers	Per-service endpoints	Controllers enforce role checks via AdminGuard using X-User-Role header

Observability and Error Handling

- Gateway ensures X-Request-Id exists (generates UUID if missing) and returns it in responses.
- Services attach X-Request-Id to responses and store it in MDC as requestId.
- Services use a shared GlobalExceptionHandler returning a structured ApiError including requestId.
- Domain actions are recorded in MongoDB collection audit_logs via the common module AuditService.
- Monitoring stack (Prometheus, Grafana, Loki, Fluent Bit) installed via Helm/GitOps scripts.

Deployment Strategy (Manual GitOps)

Deployments are applied from the gitops/ directory using a scripted 'manual GitOps' workflow with Kustomize overlays. Two overlays exist: local (Kind cluster, imagePullPolicy: IfNotPresent) and production (GKE, GCR images, GCE ingress).

- Each microservice is deployed as a Kubernetes Deployment + Service.
- Resource requests: 100m CPU / 256Mi; limits: 500m CPU / 512Mi.
- api-gateway: NodePort on port 80 (target 8080). All other services: ClusterIP on port 8080.
- HPAs exist but currently configured with minReplicas=maxReplicas=1 (no active scaling).
- Secrets consumed via Kubernetes Secret named carsharing-secrets (MONGODB_URI, JWT_SECRET_KEY, etc.).

NOTE: The repository currently contains gitops/base/secrets.yaml with base64-encoded values. Base64 is not encryption. Real secret values should not be committed to Git — prefer Sealed Secrets or External Secrets.

2.2 D2 — Technology Inventory

Backend — Core Runtime Technologies

Category	Technology	Version	Purpose
Language	Java	21	Primary backend language
Framework	Spring Boot	3.2.3	Microservice foundation
Cloud	Spring Cloud	2023.0.0	Gateway, Kubernetes integration, load balancer
Auth	JJWT	0.11.5	JWT issuance and validation
Routing API	OSRM	public endpoint	Route geometry, distance, duration enrichment

Category	Technology	Version	Purpose
DB	MongoDB Atlas	shared instance	Persistence for all services
Notifications	Firebase Admin SDK	9.2.0	Push notifications
Trip integrations	Google Maps Services	2.2.0	Trip route enrichment
Observability	Micrometer / Prometheus	via Spring Boot	Metrics scraping at /actuator/prometheus
Tracing	Micrometer / OTel / OTLP	via parent POM	Distributed tracing
Logging	logstash-logback-encoder	7.4	Structured JSON logging for Loki
API docs	springdoc-openapi	2.3.0	OpenAPI / Swagger UI per service
Codegen	Lombok	1.18.42	Boilerplate reduction
Testing	Mockito (inline)	5.11.0	Unit test mocking

Frontend — Core Runtime Technologies

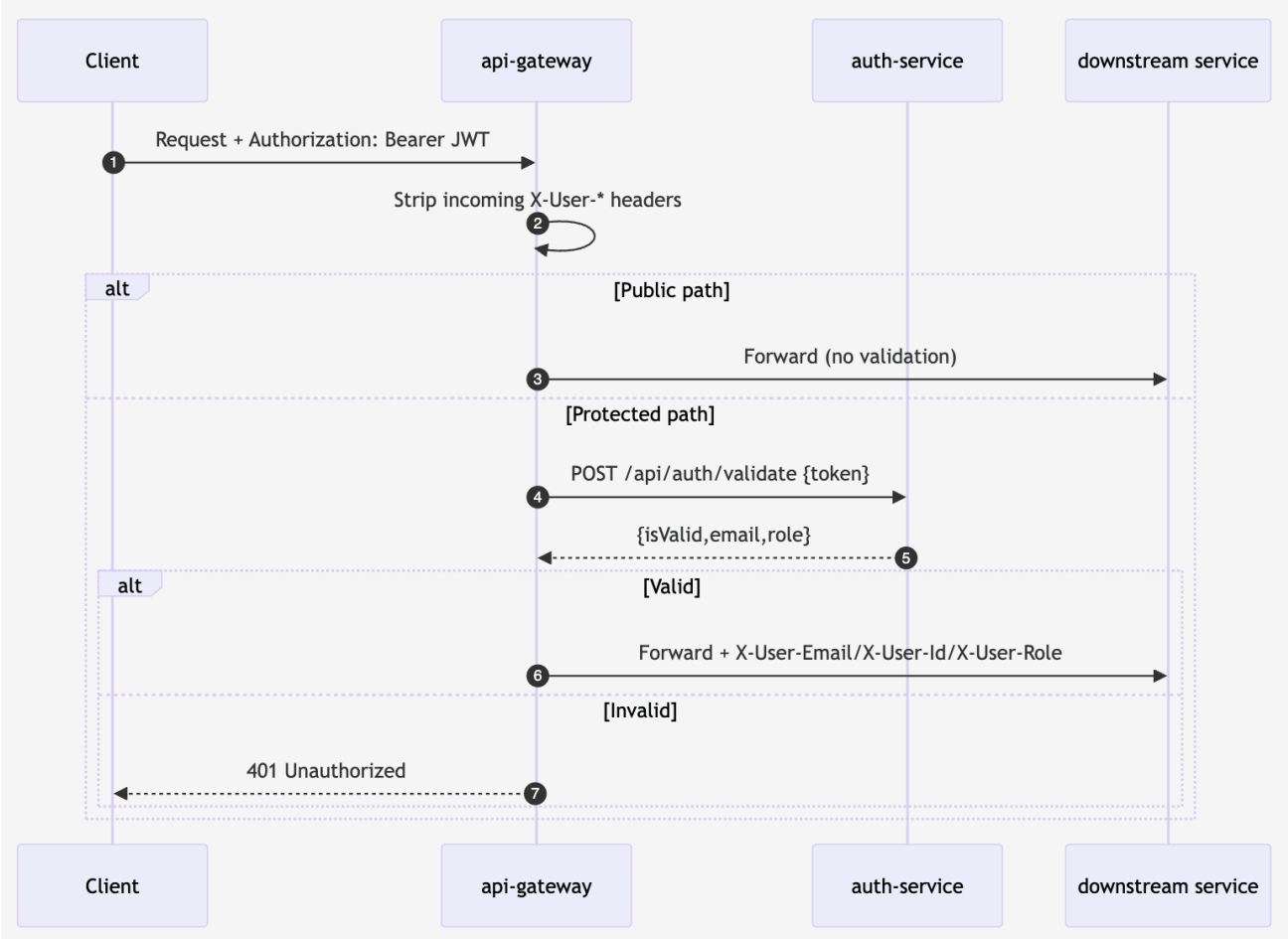
Technology	Version	Purpose
Flutter	SDK ^3.8.1	Cross-platform UI framework
Dart	SDK ^3.8.1	Programming language
Dio	^5.4.3	HTTP client for REST communication with interceptors
Flutter Riverpod	^2.6.1	State management and dependency injection
Riverpod Annotation	^2.3.3	Code generation support for Riverpod
Flutter Secure Storage	^9.1.1	Encrypted local storage of tokens
json_annotation / json_serializable	^4.9.0 / ^6.7.1	Serialization annotations + generated serialization
flutter_map	^6.0.0	Map rendering using OpenStreetMap
geolocator	^10.0.0	Device location access
permission_handler	^11.4.0	Runtime permission handling
intl	^0.19.0	Date and time formatting
email_validator	^2.1.17	Email validation
confetti	^0.7.0	Booking confirmation animation

Coding Conventions (Frontend)

Convention	Rule
File names	snake_case.dart
Class names	UpperCamelCase
Methods and variables	lowerCamelCase
Private members	prefixed with _
Async handling	async / await
Null safety	Full Dart null safety enforced
Import order	Dart SDK, Flutter SDK, third-party packages, local project imports

2.3 D3 — Detailed Design

Backend — Gateway: Authentication and Identity Propagation



The gateway acts as a centralized authentication point. All external traffic flows through the api-gateway.

6. Client sends Request + Authorization: Bearer JWT.

7. Gateway strips incoming X-User-* headers.
8. For public paths: request is forwarded directly without validation.
9. For protected paths: Gateway calls POST /api/auth/validate {token} on auth-service.
10. auth-service returns {isValid, email, role}.
11. If valid: Gateway forwards request with X-User-Email, X-User-Id, X-User-Role headers injected.
12. If invalid: Gateway returns 401 Unauthorized to client.

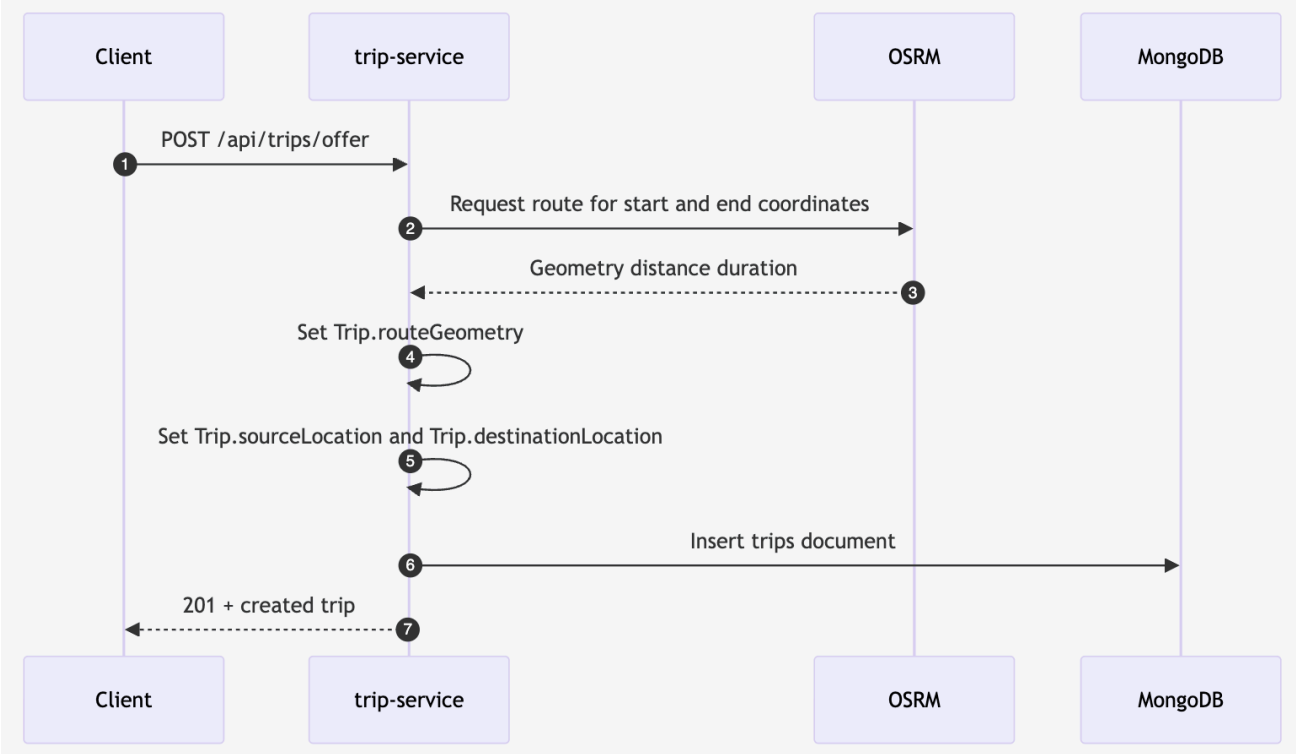
Design note: X-User-Id is currently set to the normalized email (email.toLowerCase()), not a stable opaque identifier. The codebase contains a BlacklistedToken model but validation does not consult it; logout/blacklisting is not fully implemented end-to-end.

Backend — Signup Cross-Service Flow

Signup is cross-service and attempts to keep data consistent by rolling back the auth user record if profile creation fails.

13. Client sends POST /api/auth/signup to auth-service.
14. auth-service inserts user document in MongoDB.
15. auth-service calls POST /api/users/profile on user-service.
16. If profile created (201): auth-service returns 201 + JWT to client.
17. If profile creation fails: auth-service deletes the auth user record (rollback), returns 5xx to client.

Backend — Trip Offer, Routing, and Geo Enrichment

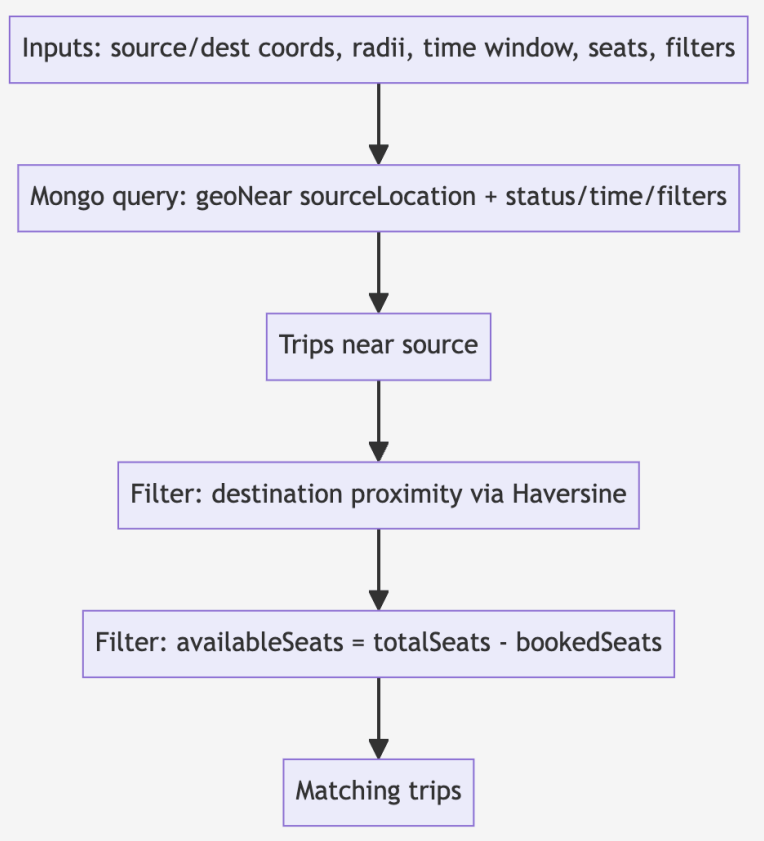


When a driver offers a ride, the service calls OSRM to obtain the route, stores route metadata, and sets geospatial fields (sourceLocation, destinationLocation) to enable location-based search.

18. Client sends POST /api/trips/offer to trip-service.
19. trip-service requests route geometry for start/end coordinates from OSRM.
20. OSRM returns geometry, distance, and duration.
21. trip-service sets Trip.routeGeometry, Trip.sourceLocation, Trip.destinationLocation.
22. trip-service inserts trips document in MongoDB.
23. trip-service returns 201 + created trip to client.

Note: OSRM encodes coordinates as lon,lat;lon,lat in the driving path segment and the implementation requests full geometry in GeoJSON. Trip.routeGeometry is stored as a String — a 2dsphere index on this field will fail unless it is changed to a GeoJSON BSON document.

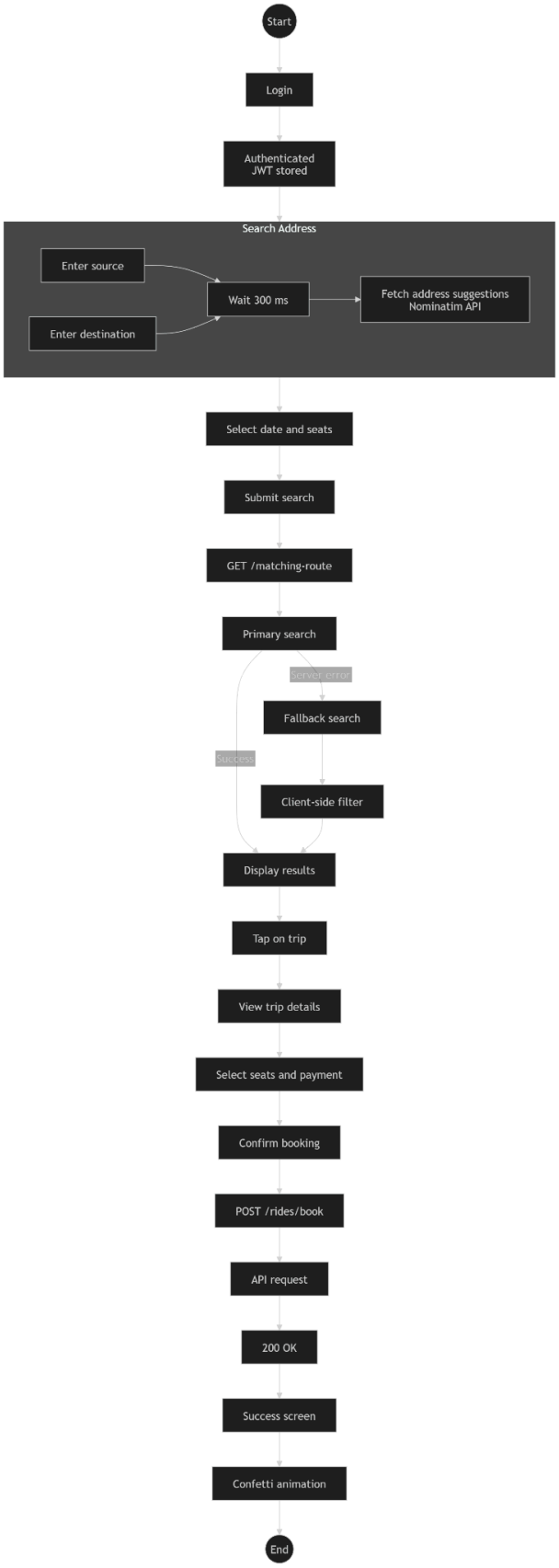
Backend — Matching-Route Search Strategy



MongoDB does not allow multiple $geoNear stages in one query, so the implementation searches near the source location in MongoDB and then filters destination proximity in memory.

24. Input: source/dest coordinates, radii, time window, seats, filters.
25. MongoDB query: geoNear on sourceLocation + status/time/filters.
26. In-memory filter: destination proximity via Haversine.
27. In-memory filter: availableSeats = totalSeats - bookedSeats.
28. Output: matching trips returned to client.

Frontend — DioClient (Networking)


Start
Login
Authenticated JWT stored
Search Address
Enter source
Enter destination
Wait 300 ms
Fetch address suggestions Nominatim API
Select date and seats
Submit search
GET /matching-route
Primary search
Server error
Fallback search
Success
Client-side filter
Display results
Tap on trip
View trip details
Select seats and payment
Confirm booking
POST /rides/book
API request
200 OK
Success screen
Confetti animation
End

The DioClient serves as the central network entry point for authenticated and unauthenticated HTTP requests.

- Stores the base backend URL.
- Defines connection, send, and receive timeouts.
- Injects the JWT token into authenticated requests.
- Excludes explicitly public endpoints from token injection.
- Handles authentication failure centrally.
- Normalizes error handling.

Behavior: Before an authenticated request is sent, the interceptor reads the token from secure storage. If the token exists, the request receives an Authorization header. If a response returns HTTP 401, locally stored credentials are removed and the app redirects the user back to login.

Frontend — TokenStorage

TokenStorage wraps flutter_secure_storage and provides named methods for reading, writing, and deleting session-related values: access token, user identifier, role, and other session metadata. This abstraction isolates platform-secure storage from the rest of the application.

Frontend — Client Services

Service	Responsibilities
Authentication Service	Registration, login, logout-related token handling. During login: sends credentials, receives JWT, stores token securely, extracts user info from token payload.
Trip Service	Search for available trips, create a trip, cancel a trip, fetch driver trips, retrieve searchable trip data for passengers. Includes fallback behavior when primary route-matching endpoint is unavailable.
Booking Service	Creating a booking, retrieving passenger bookings, canceling a booking. Cash is the currently implemented payment option.
User & Vehicle Service	Profile-related functionality and vehicle registration. No formal vehicle ownership or driver-license verification implemented yet.
Location & Map Service	Address search through autocomplete, location selection on map, map-based trip context, retrieval of current device location, conversion between place descriptions and coordinate data.

Frontend — State Management (Riverpod)

Riverpod is used to fetch asynchronous data, expose loading/data/error state to the UI, separate business logic from widgets, and connect services to presentation code. This reduces direct coupling between screens and backend logic.

Frontend — Route Search Fallback

If the primary trip route matching endpoint is unavailable, the system uses an alternative search strategy based on source proximity and client-side filtering. This improves resilience and allows trip search to remain available under partial backend failure conditions.

Standardized API Responses and Errors (Backend)

Success and error bodies are standardized across services using a shared GlobalExceptionHandler returning a structured ApiError that includes requestId for traceability. Error codes are centralized in common module ErrorCode enum. The wrapper DTO ApiResponseWrapper provides a consistent envelope for all API responses.

2.4 D6 — Data Model

Backend — Storage Strategy

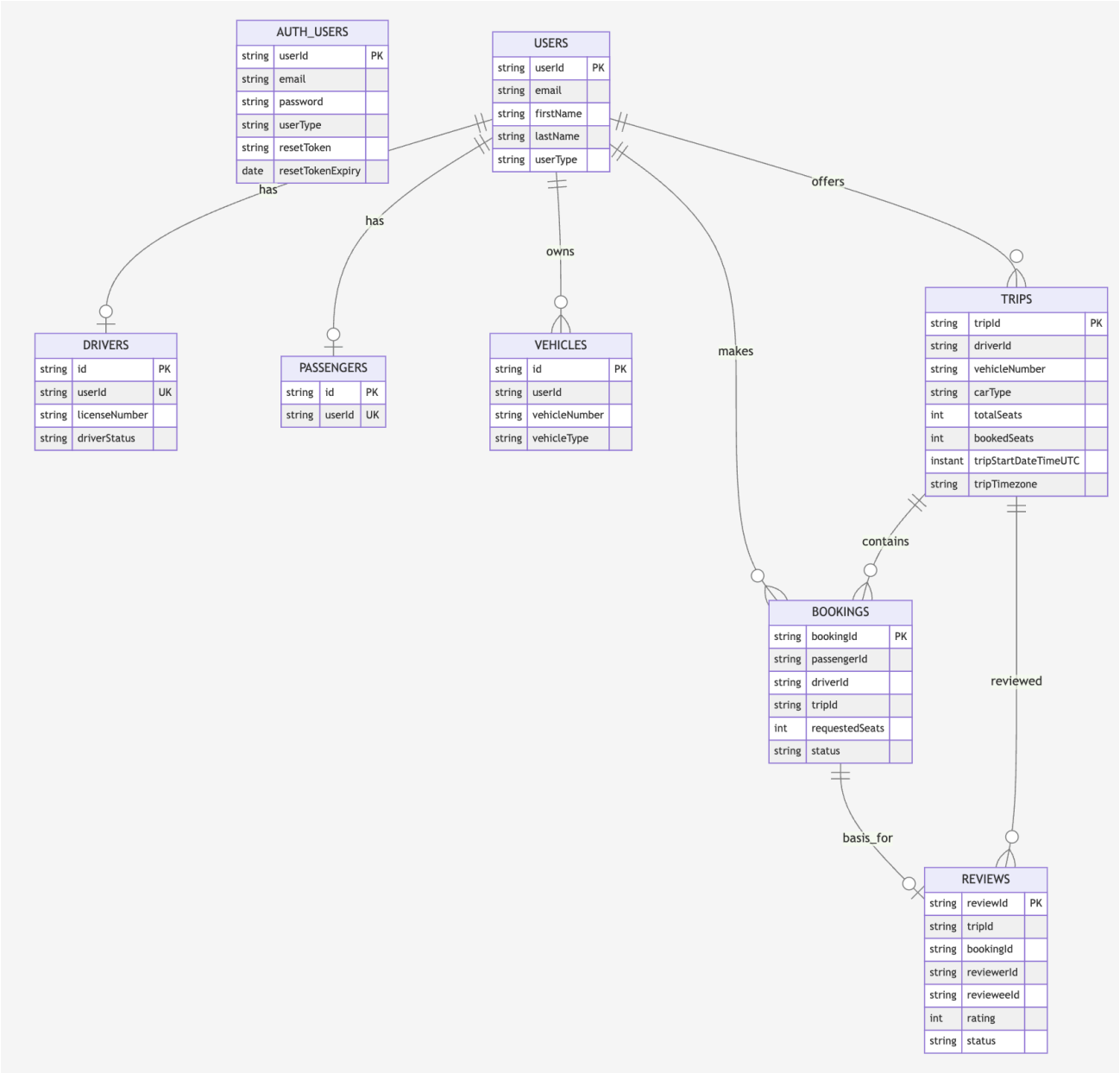
The backend uses MongoDB Atlas. The current approach is a shared MongoDB instance across services, with collections separated by domain. Services consume MONGODB_URI via environment variables sourced from Kubernetes Secrets.

Backend — Collection Overview

Domain	Collection	Owner Module	Document Class
Auth	users	auth-service	auth-service/.../model/User.java
Auth	blacklisted_tokens	auth-service	auth-service/.../model/BlacklistedToken.java
User	users	user-service	user-service/.../model/User.java
User	drivers	user-service	user-service/.../model/Driver.java
User	passengers	user-service	user-service/.../model/Passenger.java
User	vehicles	user-service	user-service/.../vehicle/model/Vehicle.java
Trip	trips	trip-service	trip-service/.../model/Trip.java
Trip	bookings	trip-service	trip-service/.../model/Booking.java
Review	reviews	review-service	review-service/.../model/Review.java
Common	audit_logs	common	common/.../model/AuditLog.java

Backend — Logical Data Model (Cross-Collection Relationships)

AUDIT_LOGS		
string	id	PK
string	action	
string	entityType	
string	entityId	
string	performedBy	
string	details	
string	serviceName	
instant	timestamp	



Backend — Indexing Strategy

- Trip collection stores sourceLocation and destinationLocation as Spring Data Point with @GeoSpatialIndexed(GEO_2DSPHERE).
- Index creation enforced at startup via MongoConfig: sourceLocation_2dsphere, destinationLocation_2dsphere.
- drivers.userId: unique index (@Indexed(unique = true)).
- passengers.userId: unique index.

RISK: Trip.routeGeometry is a String containing serialized GeoJSON, but MongoConfig attempts to create a 2dsphere index on routeGeometry. If route-based geospatial queries are planned, routeGeometry should be stored as a GeoJSON BSON object rather than a string.

Frontend — Client-Side Data Model

Kamili Drive is a client application and does not directly own the backend database schema. This section documents the logical client-side data model and its mapping to backend JSON payloads.

Core Entities

Entity	Main Attributes	Notes
User	userId, name, email, phoneNumber, age, role	Represents an authenticated application user. A driver may own multiple vehicles, create multiple trips. A passenger may create multiple bookings.
Vehicle	vehicleId, ownerUserId, make, model, licensePlate, vehicleType, seatingCapacity	Registered by the driver; no formal verification workflow implemented yet.
Location	latitude, longitude, placeAddress	Used for source location, destination location, map selection, geocoding results.
Trip	tripId, driverId, vehicleId, sourceAddress/Location, destinationAddress/Location, tripStartDateTime, availableSeats, tripStatus	One trip belongs to one driver; may be associated with one vehicle; may have multiple bookings.
Booking	bookingId, tripId, passengerId, seatCount, paymentMethod, status, timestamps	One booking belongs to one passenger and one trip.

Logical Relationships Summary

Entity	Related To	Relationship
User (Driver)	Vehicle	one-to-many
User (Driver)	Trip	one-to-many
User (Passenger)	Booking	one-to-many
Trip	Booking	one-to-many
Trip	Location	source and destination use Location objects

Entity	Related To	Relationship
Vehicle	Trip	one-to-many or reference-based, depending on backend payload

Client models are mapped from backend JSON responses using generated serialization methods. Custom converters handle date/time formatting, location conversion, and nested API objects.

3. Quality Assurance

3.1 E1 — Test Plan

Frontend — Testing Strategy

Kamili Drive uses a combination of unit tests, widget tests, integration tests, and manual exploratory testing.

Test Levels

Level	Goal	Tooling / Scope
Unit Tests	JSON serialization/deserialization, request formatting, coordinate and date conversion, route distance calculations, non-UI service logic.	Dart test framework, flutter test
Widget Tests	Basic rendering of application screens, root widget behavior, loading and interaction behavior for selected widgets.	flutter test, WidgetTester
Integration Tests	End-to-end communication with the live backend, request/response compatibility, trip search, trip creation, route filtering behavior.	flutter test integration_test/
Manual Testing	Registration/login, booking and cancellation flows, driver trip creation and withdrawal, map interaction, current-location retrieval, permission handling, UI under delayed/failed requests.	Exploratory, device testing

Frontend — Coverage Goals

- Target coverage: approximately 70% of core logic in lib/core/ and lib/features/*/data/.
- Primary focus: service logic, serialization, request building, and route-search behavior.
- Lower coverage acceptable in purely visual UI code.

Frontend — Planned Test Areas

- Authentication requests and token flow.
- Trip search parameters and fallback route logic.
- Booking creation and cancellation.
- Driver trip creation and trip withdrawal.
- Vehicle registration data validation.
- Map-based location selection and current-location retrieval.
- Error handling for invalid backend responses.
- Timeout and unavailable-server behavior.

Backend — Test Levels and Scope

Level	Goal	Tooling (Intended)	Current Status
Unit tests	Validate core business logic in isolation	JUnit 5, Mockito	Present in auth-service only
Integration tests	Validate service boundaries (DB, HTTP clients)	Spring Boot Test, Testcontainers (Mongo)	Not present in repo
Contract/API tests	Validate request/response contracts	Springdoc + schema checks; black-box tests	Not present in repo
E2E (local k8s)	Validate user journeys across services	Kind + GitOps scripts + cURL	Manual examples only

Backend — Coverage Goals

- Unit test coverage target: >= 70% line coverage for pure service-layer code.
- Critical paths: JWT validation and claims extraction (JwtService), signup cross-service profile creation and rollback (AuthService.registerUser), trip offer flow including OSRM failure handling (OfferRideService), trip matching-route query logic (TripGeospatialService), admin authorization enforcement.
- No coverage report currently produced — adding JaCoCo to Maven build is a follow-up task.

3.2 E2 — Test Report

Frontend — Test Execution

Tests were executed using flutter test for unit and widget tests, and flutter test integration_test/ for integration tests against the live backend.

Frontend — Unit and Widget Test Results

Metric	Result	Notes
Total tests	14	Unit + Widget tests combined
Passed	14	All passing
Failed	0	—
Static Analysis Errors (flutter analyze)	0	0 warnings, some info/hints in test code
Coverage (estimated)	~65%	Slightly below 70% target; sufficient for educational prototype

Frontend — Existing Automated Test Cases

File	Test Name	Description	Type
test/widget_test.dart	App renders welcome screen	Verifies application starts and renders initial widget tree	Widget

File	Test Name	Description	Type
test/api_request_test.dart	Trip search parameters should be numeric types	Confirms numeric parameter formatting	Unit
test/api_request_test.dart	OfferRideRequest should serialize correctly	Checks field names and JSON types	Unit
test/api_request_test.dart	Timestamp format should be ISO 8601 UTC	Verifies proper UTC serialization	Unit
test/models/booking_response_test.dart	DriverTripResponse uses passengers field	Verifies correct mapping of booking-related backend response data	Unit
test/route_matching_service_test.dart	Paris to London distance ~343 km	Verifies Haversine distance implementation	Unit
integration_test/api_integration_test.dart	Trip search with valid parameters	Verifies live route search response	Integration
integration_test/api_integration_test.dart	Trip search with no results	Verifies empty search handling	Integration
integration_test/api_integration_test.dart	Near-source search returned trips within radius	Checks client-side route proximity logic	Integration
integration_test/api_integration_test.dart	Trip creation with valid request	Verifies live trip creation request behavior	Integration

Frontend — Integration Test Results

Test	Result	Notes
Trip search — valid parameters	PASS	Returned a list with expected trip structure
Trip search — no results	PASS	Returned an empty list correctly
Near-source search radius check	PASS	Returned trips satisfied distance tolerance
Trip creation	PASS*	*Requires valid driver JWT; may fail with 401 in CI or invalid-auth environments
Driver active trips retrieval	PASS	Returned correct structure, including empty result cases

Backend — Tests Present in Repository

Module	Test File	Type	Coverage
auth-service	AuthServiceApplicationTests.java	Spring context	Application context loads

Module	Test File	Type	Coverage
auth-service	AuthServiceResetPasswordTest.java	Unit (Mockito)	Password reset token generation and confirmation

Backend — Notable Gaps

- No tests for api-gateway filters (JWT enforcement, admin filter, request-ID propagation).
- No integration tests for MongoDB persistence or geospatial queries.
- No contract/e2e test suite tracked in CI.

Frontend — Test Areas Still Incomplete

- Broader widget coverage for booking and trip flows.
- Automated testing of map interaction.
- Automated testing of location permission denial cases.
- Automated testing of booking cancellation edge cases.
- Automated testing of trip withdrawal edge cases.
- Future payment integration tests once payment gateways are added.

Frontend — Limitations of Current Testing

- Some integration tests depend on the availability and state of the live backend.
- Authentication-dependent tests require valid tokens and suitable backend data.
- Map and GPS interactions are currently tested primarily through manual testing.
- Only cash payment is implemented — no real payment gateway tests exist yet.

4. Final Report

4.1 A2 — Summary

Kamili Drive is a Flutter-based car-sharing application prototype that connects drivers offering rides with passengers looking for trips. The current system supports the main passenger and driver workflows required for a functional prototype, backed by a Spring Boot microservices platform deployed on Google Kubernetes Engine.

Implemented Core Features

- User registration and login via JWT-based authentication.
- Secure local storage of authentication tokens (flutter_secure_storage).
- Trip search using source, destination, date, and seat count.
- Address autocomplete and geocoding via OpenStreetMap Nominatim.
- Map display, map-based location selection, and current-location retrieval.
- Passenger booking flow with cash payment.
- Passenger booking cancellation before trip date.
- Driver vehicle registration without formal verification.
- Driver trip creation, trip listing, and trip withdrawal/cancellation.
- Backend microservices: auth, user, trip, review, notification — all behind Spring Cloud Gateway.
- OSRM-based route enrichment (geometry, distance, duration).
- MongoDB geospatial indexing and proximity-based trip search.
- Immutable audit logging, Prometheus metrics, distributed tracing, and structured logging.

Main Technical Achievements

- Migration from Firebase-based dependencies to REST-based backend integration.
- Centralized HTTP handling through Dio (client) and Spring Cloud Gateway (backend).
- Role-based passenger and driver flows with RBAC at gateway and service level.
- Generated JSON serialization (json_serializable / Lombok).
- Route-search fallback logic for improved resilience on both client and backend.
- Integration of map and location features into the client workflow.
- Microservices architecture with manual GitOps deployment on Kubernetes / GKE.

Current Limitations

- No digital payment gateway integration yet (only cash supported).
- No formal vehicle or driver-license verification workflow.
- Desktop compilation exists, but mobile remains the primary supported target.
- Some advanced testing scenarios are still incomplete.
- Backend secret management: secrets.yaml committed with base64-encoded values.
- Token blacklisting model exists but is not consulted during validation (logout incomplete).

- Ingress port mismatch between api-gateway Service (port 80) and local ingress backend (8080).

Future Work

- Tanzanian payment gateway integration, such as AzamPay.
- Formal verification workflows for vehicles or licenses.
- Stronger automated UI and map testing; broader production hardening.
- Backend-contract stabilization; add JaCoCo coverage reporting.
- Resolve routeGeometry index mismatch (String vs GeoJSON BSON).
- Implement proper logout/token blacklisting end-to-end.
- Align Ingress service port with api-gateway Service definition.

4.2 E3 — Example Application

Example Scenario: Driver Offers a Trip and Passenger Books It

Step 1: Driver Login

A driver logs into the application using email and password. The backend auth-service validates credentials and returns a JWT token. The application stores this token securely and uses it for subsequent authenticated requests.

Step 2: Vehicle Registration

If the driver has not yet added a vehicle, they register one by entering: make/model, license plate, vehicle type, and seating capacity. The vehicle becomes available for use when creating trips. In the current version, this is a user-entered registration process and does not include official verification.

Step 3: Driver Creates a Trip

The driver opens the trip creation screen, selects source and destination either by search or map interaction, chooses the departure time, selects a registered vehicle, and publishes the trip. The client sends POST /api/trips/offer to the backend trip-service, which enriches the trip via OSRM, stores it in MongoDB, and returns a success response.

Step 4: Passenger Searches for a Trip

A passenger enters a source, destination, travel date, and seat count. The application provides address suggestions via Nominatim and supports map-based location interaction. The passenger submits the search; the client calls GET /matching-route on the backend, which performs a geospatial query on MongoDB and returns matching trip results. If the primary endpoint is unavailable, the fallback search strategy is used.

Step 5: Passenger Books the Trip

The passenger selects a suitable trip and confirms the booking. The current payment option is cash. The client sends POST /rides/book to the backend. After a successful 200 OK response, the application displays a confirmation screen with confetti animation.

Step 6: Trip and Booking Management

- The passenger can view booked trips and cancel a booking before the trip date.

- The driver can view their created trips and withdraw a trip if necessary.

This scenario demonstrates the complete core flow currently supported by the prototype.

4.3 F1 — User Documentation

Supported Platforms

- Android 5.0+
- iOS 13.0+
- Flutter desktop targets available for development and testing.

Installation for Development and Testing

```
# Clone the repository
git clone <repository-url>

# Fetch dependencies
flutter pub get

# Generate serialization and provider code
dart run build_runner build --delete-conflicting-outputs

# Run the application
flutter run
```

Basic Usage

Registration

A new user creates an account by entering personal information and choosing a role: passenger or driver.

Login

The user signs in with email and password. After successful authentication, the application routes the user into the correct role-specific flow.

Passenger Flow

- Search for a trip using source, destination, date, and seat count.
- Use autocomplete and map interaction for location selection.
- Retrieve current location.
- View matching trips.
- Book a trip using cash payment.
- View current and past bookings.
- Cancel a booking before the trip date.

Driver Flow

- Add a vehicle.

- Create a trip.
- Choose locations using search or map interaction.
- View published trips.
- Inspect seat usage.
- Withdraw a trip.

Known Functional Limitations

- Only cash payment is supported.
- No official vehicle verification is available yet.
- Some workflows depend on live backend availability.

4.4 F2 — Developer Documentation

Frontend — Project Structure

Module	Contents
lib/core/	Shared networking (DioClient), storage (TokenStorage), utilities
lib/features/auth/	Authentication models, services, UI
lib/features/trip/	Trip search, creation, listing
lib/features/booking/	Booking creation and cancellation
lib/features/user/	Profile and vehicle features
lib/features/map/	Map and location-related functionality

Frontend — Adding a New API Endpoint

29. Add the method to the correct service class.
30. Reuse the centralized Dio client rather than making isolated network calls.
31. Create or update model classes for request and response payloads.
32. Generate serialization code if required.
33. Expose the feature through a Riverpod provider or notifier if UI state depends on it.
34. Update the relevant screen or widget and handle loading, success, and failure states.

Frontend — Adding a New Data Model

35. Create a Dart model class using serialization annotations.
36. Add required converters for nested or custom types such as locations or timestamps.
37. Run code generation: dart run build_runner build --delete-conflicting-outputs
38. Add tests for serialization and backend mapping.

Frontend — Architectural Rules for Contributors

- Keep business logic out of widgets.
- Centralize authenticated requests through Dio client.

- Prefer typed models instead of dynamic JSON handling.
- Document public service methods.
- Add tests for serialization, request construction, and important user flows where feasible.

Backend — Repository Structure

Module	Contents
api-gateway/	Spring Cloud Gateway, JWT filter, admin filter, request-ID filter, routing config
auth-service/	AuthController, AuthService, JwtService, UserServiceClient, rollback on signup
user-service/	User, Driver, Passenger profiles; Vehicle model; admin controllers
trip-service/	OfferRideService, TripGeospatialService, Trip, Booking models; admin + reports controllers
review-service/	Review model, admin review controller
notification-service/	Firebase-based push notifications
common/	ApiResponseWrapper, GlobalExceptionHandler, AuditService, RequestIdFilter, AdminGuard, ErrorCode
gitops/	Kubernetes manifests (base + overlays), GitOps scripts, monitoring Helm setup

Backend — Admin Endpoint Inventory

Service	Base Path	Controller
user-service	/api/admin/drivers	AdminDriverController.java
user-service	/api/admin/passengers	AdminPassengerController.java
user-service	/api/admin/vehicles	AdminVehicleController.java
trip-service	/api/admin/trips	AdminTripController.java
trip-service	/api/admin/bookings	AdminBookingController.java
trip-service	/api/admin/reports	ReportsController.java (incl. audit-logs query)
review-service	/api/admin/reviews	AdminReviewController.java

5. Code Documentation Standards

The Kamili Drive codebase follows standard Dart and Flutter documentation practices on the client side, and standard Java/Javadoc conventions on the backend.

Documentation Principles

- Each module should have a clear purpose and descriptive naming.
- Public classes and public service methods should include Dartdoc (frontend) or Javadoc (backend) comments.
- Complex logic should be supported by inline comments where necessary.
- Names for classes, fields, methods, and variables should be explicit and meaningful.

Applied Practice

Particular attention has been given to documenting:

- Network service methods (DioClient, auth/trip/booking services).
- Token and authentication handling.
- Custom serialization and conversion logic.
- Route-search fallback logic.
- Model classes that map backend payloads.
- Backend: JWT validation and claims extraction, audit logging design, admin authorization.

Naming and Readability

Context	Convention	Example
Flutter — Classes	UpperCamelCase	TripService, BookingResponse
Flutter — Variables/Methods	lowerCamelCase	availableSeats, fetchTrips()
Flutter — Files	snake_case.dart	trip_service.dart, booking_response.dart
Flutter — Private members	prefixed with _	_token, _loadData()
Java — Classes	UpperCamelCase	TripGeospatialService, AdminGuard
Java — Methods/variables	lowerCamelCase	offerRide(), requestedSeats
Java — Packages	lowercase.dotted	com.cimhub.tripservice.service

6. Technical Retrospective

What Worked Well

1. REST-Based Microservices Architecture

The separation between authentication, trip, user, and review functionality at the backend level made responsibilities easier to understand and integrate from the client side. Spring Cloud Gateway provided a clean centralized entry point.

2. Riverpod for State Management

Riverpod provided a clean way to model asynchronous states such as loading, success, and error. This improved UI consistency and helped keep business logic outside widget code.

3. Code Generation

Using generated serialization (json_serializable on the frontend, Lombok on the backend) reduced manual mapping errors and simplified maintenance of request and response models.

4. Centralized Networking

The use of a single Dio-based network client on the frontend, and the Spring Cloud Gateway on the backend, simplified authentication, timeout handling, error handling, and backend communication.

5. Map and Location Integration

Integrating OpenStreetMap search, map interaction, and current-location retrieval improved the usability of trip creation and trip search flows.

6. GitOps Deployment Discipline

Maintaining a manual GitOps rule (no direct kubectl edit) and using Kustomize overlays for local vs. production environments provided a structured and reproducible deployment workflow.

What Was Challenging

1. API Contract Alignment

Backend integration required repeated checking of endpoint paths and response shapes. Some effort was lost due to mismatches between expected and actual backend structures.

2. Live Integration Dependency

Because integration tests use the real backend, test stability can be affected by backend availability, data state, and valid authentication context.

3. Scope of Advanced Features

Payment-gateway integration and formal verification workflows require external coordination and domain-specific knowledge that were not yet available during this development phase.

4. MongoDB Multi-Geospatial Limitation

MongoDB does not allow multiple $geoNear stages in one query, which required a hybrid approach: MongoDB query near source, followed by in-memory Haversine filtering for destination proximity.

What We Would Do Differently

1. Validate Backend Responses Earlier

A stronger habit of inspecting real endpoint responses before implementing models would reduce rework.

2. Formalize API Contracts Earlier

Closer backend-frontend contract stabilization earlier in the project would have saved debugging time.

3. Expand Automated UI and Location Testing Sooner

Map and location features were implemented successfully, but broader automated coverage should have been introduced earlier.

4. Implement Proper Secret Management from the Start

Using Sealed Secrets or External Secrets from the beginning would have avoided the current situation of base64-encoded secrets in the repository.

Outlook

The prototype now provides a stable foundation for the core driver and passenger workflows. The next logical steps are:

- Stabilizing backend contracts further.
- Extending automated tests and introducing JaCoCo coverage reporting.
- Integrating Tanzanian payment infrastructure (AzamPay).
- Evaluating feasible verification workflows for vehicles or licenses in the Tanzanian legal context.
- Implementing proper logout/token blacklisting end-to-end.
- Resolving the routeGeometry index mismatch and the Ingress port alignment issue.

Appendix: Key References

Backend Documentation

- Architecture: .docs/technical/crucial/architecture.md
- API reference: .docs/technical/crucial/api-reference.md
- API guide: .docs/technical/crucial/api-guide.md
- User flows: .docs/product/user-flows.md
- GitOps: .docs/technical/devops/gitops.md, gitops/gitops.sh
- Monitoring: .docs/technical/devops/monitoring.md, gitops/monitoring/*
- Audit logging: .docs/technical/crucial/audit-logging.md
- GCP deployment: .docs/technical/devops/gcp-deployment.md

Backend Key Source Files

- Gateway: api-gateway/src/main/java/.../JwtValidationFilter.java, AdminAuthorizationFilter.java, RequestIdGatewayFilter.java
- Auth: auth-service/src/main/java/.../AuthService.java, JwtService.java
- Trip: trip-service/src/main/java/.../OfferRideService.java, TripGeospatialService.java
- Common: common/src/main/java/.../AuditService.java, GlobalExceptionHandler.java, AdminGuard.java
- GitOps manifests: gitops/base/deployments/*.yaml, gitops/base/rbac/spring-cloud-rbac.yaml

Frontend Repositories

- Flutter client: carsharing-frontend (Flutter/Dart)
- Backend: carsharing-backend (Java/Spring Boot multi-module Maven)